

2nd Week

Numpy 소개





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 lpython – 파이썬에 날개를 달자	실습 과제 (채점 X)
02주차	Numpy 소개	실습 과제
03주차	Pandas로 데이터 가공하기	실습 과제
04주차	Matplotlib를 활용한 시각화	실습 과제
05주차	탐색적 데이터 분석	실습 과제
06주차	데이터의 표본분포	실습 과제
07주차	통계적 실험과 유의성 검정	실습 과제
08주차	중간고사 – 오프라인 오픈북으로 진행	기말 프로젝트 상세 공지

기간	내용	과제
09주차	인공지능 & 회귀/예측	실습 과제
10주차	회귀와 예측 / 분류	실습 과제
11주차	분류 및 머신러닝 기초	실습 과제
12주차	통계적 머신러닝	
13주차	비지도 학습	실습 과제
14주차	알고보면 쓸모있는 신기한 기계학습	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!



Jupyter Notebook 단축키

Keyboard shortcuts of Jupyter Notebook

Keyboard shortcuts

The Jupyter Notebook has two different keyboard input modes. **Edit mode** allows you to type code/text into a cell and is indicated by a green cell border. **Command mode** binds the keyboard to notebook level actions and is indicated by a grey cell border with a blue left margin.

Command Mode (press `Esc` to enable)

`F`: find and replace
`Ctrl-Shift-P`: open the command palette
`Enter`: enter edit mode
`Shift-Enter`: run cell, select below
`Ctrl-Enter`: run selected cells
`Alt-Enter`: run cell, insert below
`Y`: to code
`M`: to markdown
`R`: to raw
`1`: to heading 1
`2`: to heading 2
`3`: to heading 3
`4`: to heading 4
`5`: to heading 5
`6`: to heading 6
`K`: select cell above
`Up`: select cell above
`Down`: select cell below
`J`: select cell below

Edit Mode (press `Enter` to enable)

`Tab`: code completion or indent
`Shift-Tab`: tooltip
`Ctrl-]`: indent
`Ctrl-[`: dedent
`Ctrl-A`: select all
`Ctrl-Z`: undo
`Ctrl-Shift-Z`: redo
`Ctrl-Y`: redo
`Ctrl-Home`: go to cell start
`Ctrl-Up`: go to cell start
`Ctrl-End`: go to cell end
`Ctrl-Down`: go to cell end
`Ctrl-Left`: go one word left
`Ctrl-Right`: go one word right
`Ctrl-Backspace`: delete word before
`Ctrl-Delete`: delete word after
`Ctrl-M`: command mode
`Ctrl-Shift-P`: open the command palette
`Esc`: command mode

`Shift-K`: extend selected cells above
`Shift-Up`: extend selected cells above
`Shift-Down`: extend selected cells below
`Shift-J`: extend selected cells below
`A`: insert cell above
`B`: insert cell below
`X`: cut selected cells
`C`: copy selected cells
`Shift-V`: paste cells above
`V`: paste cells below
`Z`: undo cell deletion
`D`, `D`: delete selected cells
`Shift-M`: merge selected cells, or current cell with cell below if only one cell selected
`Ctrl-S`: Save and Checkpoint
`S`: Save and Checkpoint
`L`: toggle line numbers
`O`: toggle output of selected cells
`Shift-O`: toggle output scrolling of selected cells
`H`: show keyboard shortcuts
`I`, `I`: interrupt kernel
`Q`, `Q`: restart the kernel (with dialog)
`Esc`: close the pager
`Q`: close the pager
`Shift-Space`: scroll notebook up
`Space`: scroll notebook down

`Shift-Enter`: run cell, select below
`Ctrl-Enter`: run selected cells
`Alt-Enter`: run cell, insert below
`Ctrl-Shift-Minus`: split cell
`Ctrl-S`: Save and Checkpoint
`Down`: move cursor down
`Up`: move cursor up



Google Colab 단축키

Keyboard shortcuts of Google Colab

키보드 환경설정

단축키

단축키를 추가하거나 변경하려면 키 조합을 클릭한 다음 새 키를 입력하세요. Ctrl+M을(들) 다중 키-이벤트 단축키의 접두어로 사용할 수 있습니다.

단축키 설정	.ipynb 다운로드	단축키 설정	세션 다시 시작 및 모두 실행
단축키 설정	.py 다운로드	단축키 설정	섹션 제목 셀 추가
단축키 설정	Colab Enterprise 열기	단축키 설정	셀 또는 선택 항목 복사
단축키 설정	Drive에 사본 저장	단축키 설정	셀 또는 선택 항목 잘라내기
단축키 설정	Drive에서 노트북 이동	Ctrl+Click	셀 선택 도구
단축키 설정	Enable auto-run of first cell or section on any execution	Ctrl+Shift+Y	셀 작업 다시 실행
단축키 설정	GitHub Gist로 사본 저장	Ctrl+M D	셀/선택항목 삭제
단축키 설정	GitHub에 사본 저장	단축키 설정	셀로 이동
단축키 설정	Google Drive의 노트 별표표시/별표표시 해제	Shift+Enter	셀을 실행하고 다음 셀 선택
단축키 설정	JSON 노트북 보기	Alt+Enter	셀을 실행하고 새 셀 삽입
단축키 설정	Show Colab AI	단축키 설정	스크래치 셀로 복사
단축키 설정	노트 설정 열기	Ctrl+Alt+N	스크래치 코드 셀 열기
단축키 설정	노트 업로드	단축키 설정	실습 모드에서 열기
Ctrl+O	노트 열기	Ctrl+M I	실행 중단
Ctrl+P	노트 인쇄	단축키 설정	실행된 코드 기록 표시
Ctrl+S	노트 저장	Ctrl+M B	아래에 코드 셀 삽입
단축키 설정	노트 출처 보기	Ctrl+M F	양식 보기 순환
단축키 설정	노트 파일 정보 표시	단축키 설정	양식 속성 수정
단축키 설정	노트를 휴지통으로 이동	단축키 설정	양식 입력란 추가
단축키 설정	노트북 공유	단축키 설정	양식 추가
Ctrl+F9	노트의 모든 셀 실행	단축키 설정	예약된 노트북 표시
단축키 설정	다시 연결	단축키 설정	왼쪽 창을 탭으로 이동
Ctrl+M N	다음 셀	Ctrl+M A	위에 코드 셀 삽입
Shift+Down	다음 셀을 포함하도록 선택 영역을 확장합니다.	단축키 설정	의견 사이드바
Ctrl+G	다음 찾기	Ctrl+M P	이전 셀
Ctrl+Shift+]	다음 탭으로 포커스 이동	Shift+Up	이전 셀을 포함하도록 선택 영역을 확장합니다.
Alt+Enter	다중 셀 선택	Ctrl+Shift+G	이전 찾기
Ctrl+Shift+[	이전 탭으로 포커스 이동	Ctrl+Shift+[	이전 탭으로 포커스 이동

Shift+Click	다중 셀 선택
단축키 설정	단일 탭 레이아웃 보기
Ctrl+M H	단축키 표시
단축키 설정	댓글 창 열기
Ctrl+Alt+M	댓글 추가
단축키 설정	드라이브 마운트
단축키 설정	드라이브 마운트 해제
단축키 설정	드라이브에서 찾기
단축키 설정	런타임 로그 보기
단축키 설정	런타임 연결 해제 및 삭제
단축키 설정	런타임에 연결
단축키 설정	로컬 런타임에 연결
단축키 설정	리소스 보기

Ctrl+M Z 또는 Ctrl+Shift+Z	마지막 셀 작업 실행취소
단축키 설정	마지막으로 실행한 셀로 포커스 이동
단축키 설정	맞춤 GCE VM에 연결
Ctrl+Shift+P	명령 팔레트 표시
단축키 설정	모든 런타임 초기화
Ctrl+]	모든 섹션/선택된 섹션 접기
Ctrl+[	모든 섹션/선택한 섹션 펼치기
Ctrl+Shift+A	모든 셀 선택
단축키 설정	모든 출력 지우기
단축키 설정	목차 표시
단축키 설정	버전 기록 표시
Ctrl+M S	버전 저장 및 고정
단축키 설정	변수 검사기 표시
단축키 설정	비교 노트북
단축키 설정	사용자 보안 비밀 탭 열기
단축키 설정	상태 표시줄 표시/숨기기
단축키 설정	새 노트 만들기
단축키 설정	선택된 셀 병합

Ctrl+Space 또는 Tab	자동완성
Ctrl+H	전체 찾기/바꾸기
단축키 설정	접힌 섹션 레이아웃 저장
Ctrl+Shift+S	초점이 맞춰진 셀 선택
Ctrl+Enter	초점이 맞춰진 셀 실행
단축키 설정	초점이 맞춰진 셀과 다음 셀 병합
단축키 설정	초점이 맞춰진 셀과 이전 셀 병합
단축키 설정	출력 전체화면 보기
Ctrl+M O	출력 표시/숨기기
Ctrl+M -	커서에서 분할
Tab	코드 문서화(docstring) 도움말을 도구

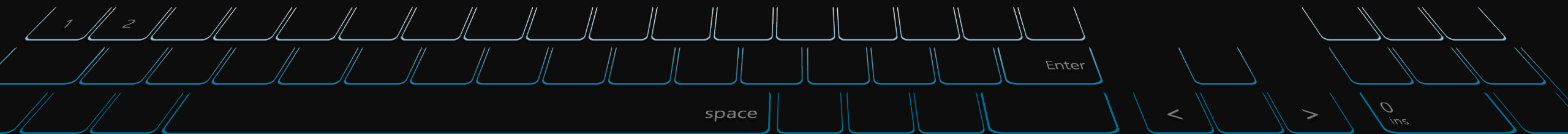
단축키 설정	코드 생성
단축키 설정	코드 셀 추가
Ctrl+M Y	코드 셀로 변환
Ctrl+Alt+P	코드 스니펫 창 표시
단축키 설정	코드 표시/숨기기
단축키 설정	탭에서 셀 미러링
단축키 설정	탭으로 포커스 이동 여부 전환
단축키 설정	탭을 다음 창으로 이동
단축키 설정	탭을 열 2개로 보기
단축키 설정	탭을 이전 창으로 이동
단축키 설정	탭을 행 2개로 보기
단축키 설정	터미널 보기
단축키 설정	텍스트 셀 추가
Ctrl+M M	텍스트 셀로 변환
단축키 설정	파일 브라우저 표시
단축키 설정	편집기 설정 열기
Ctrl+M L	행 번호 전환
단축키 설정	헤더 공개 상태 전환
Ctrl+V	현재 섹션 펼치기/접기 전환

Ctrl+F10	선택된 셀을 실행하고 그 이후 모든 셀 실행	Ctrl+]	현재 섹션 접기/접기 전환
Esc	현재 셀 선택 해제	Ctrl+F8	현재 셀 이전에 셀 실행
Ctrl+M K	선택된 셀을 위로 이동	Shift+Ctrl+H	현재 셀에서 모두 바꾸기
Ctrl+M J	선택된 셀을 아래로 이동	Shift+Tab	현재 행 들어쓰기 취소
단축키 설정	선택한 출력 지우기	Ctrl+/	현재 행 주석 처리하기
Ctrl+Shift+Enter	선택항목 실행	단축키 설정	호스팅된 런타임에 연결
단축키 설정	설정 열기		
단축키 설정	세션 관리		
Ctrl+M .	세션 다시 시작		

취소

1부: *Numpy* 기초

- 1) Python 데이터 타입
- 2) Numpy 배열의 기초 / 연산





Numpy 간단 소개

Numpy!?

- 파이썬에서 인메모리 데이터를 효과적으로 적재/저장/가공하는 기법 중 하나
- 데이터 과학을 수행하는 절차에서 Pandas와 함께
가장 근본적인 작업인 '숫자 배열을 효과적으로 저장하고 가공하는 것'을 대표하는 전문 도구
- Numerical Python의 약자이다
- Python 내장 타입인 list와 비슷하지만, 배열의 규모가 커질수록 데이터 저장 및 처리에 훨씬 더 효율적이다

=> 파이썬 기반 데이터 과학 전체 생태계의 핵심을 이루고 있다



Python 데이터 타입

Data Type in Python

데이터가 어떻게 저장되고 관리되는지 알면 효과적인 데이터 과학 및 계산을 해낼 수 있다

- Python은 동적 타이핑(Dynamic Typing)을 활용
 - C나 Java는 정적 타이핑(Statically Typing)이다
 - 정적 타이핑은 모든 변수를 명시적으로 Type을 지정해서 선언해야하지만, 동적 타이핑은 모든 변수를 타입 지정없이 활용할 수 있다

For example, in C you might specify a particular operation as follows:

```
/* C code */
int result = 0;
for(int i=0; i<100; i++){
    result += i;
}
```

While in Python the equivalent operation could be written this way:

```
# Python code
result = 0
for i in range(100):
    result += i
```

Notice the main difference: in C, the data types of each variable are explicitly declared, while in Python the types are dynamically inferred. This means, for example, that we can assign any kind of data to any variable:

```
# Python code
x = 4
x = "four"
```

Here we've switched the contents of x from an integer to a string. The same thing in C would lead (depending on compiler settings) to a compilation error or other unintended consequences:

```
/* C code */
int x = 4;
x = "four"; // FAILS
```




Python 정수 > 그냥 정수

Python Integer > Just an Integer

The standard Python implementation is written in C. This means that every Python object is simply a cleverly disguised C structure, which contains not only its value, but other information as well. For example, when we define an integer in Python, such as `x = 10000`, `x` is not just a “raw” integer. It’s actually a pointer to a compound C structure, which contains several values. Looking through the Python 3.4 source code, we find that the integer (long) type definition effectively looks like this (once the C macros are expanded):

```
struct _longobject {  
    long ob_refcnt;  
    PyTypeObject *ob_type;  
    size_t ob_size;  
    long ob_digit[1];  
};
```

A single integer in Python 3.4 actually contains four pieces:

- `ob_refcnt`, a reference count that helps Python silently handle memory allocation and deallocation
- `ob_type`, which encodes the type of the variable
- `ob_size`, which specifies the size of the following data members
- `ob_digit`, which contains the actual integer value that we expect the Python variable to represent

모든 Python 객체는 C 구조로 설계되어있다!

This means that there is some overhead in storing an integer in Python as compared to an integer in a compiled language like C, as illustrated in **Figure 2-1**.

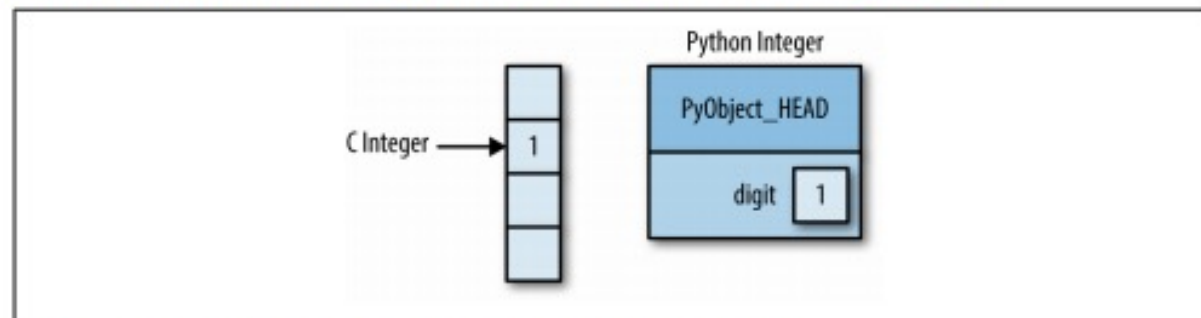


Figure 2-1. The difference between C and Python integers

Here `PyObject_HEAD` is the part of the structure containing the reference count, type code, and other pieces mentioned before.

Notice the difference here: a C integer is essentially a label for a position in memory whose bytes encode an integer value. A Python integer is a pointer to a position in memory containing all the Python object information, including the bytes that contain the integer value. This extra information in the Python integer structure is what allows Python to be coded so freely and dynamically. All this additional information in Python types comes at a cost, however, which becomes especially apparent in structures that combine many of these objects.



Python 리스트 > 그냥 리스트

Python List > Just a List

Let's consider now what happens when we use a Python data structure that holds many Python objects. The standard mutable multielement container in Python is the list. We can create a list of integers as follows:

```
In[1]: L = list(range(10))
      L
Out[1]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
In[2]: type(L[0])
Out[2]: int
```

Or, similarly, a list of strings:

```
In[3]: L2 = [str(c) for c in L]
      L2
Out[3]: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
In[4]: type(L2[0])
Out[4]: str
```

Because of Python's dynamic typing, we can even create heterogeneous lists:

```
In[5]: L3 = [True, "2", 3.0, 4]
      [type(item) for item in L3]
Out[5]: [bool, str, float, int]
```

But this flexibility comes at a cost: to allow these flexible types, each item in the list must contain its own type info, reference count, and other information—that is, each item is a complete Python object. In the special case that all variables are of the same type, much of this information is redundant: it can be much more efficient to store data in a fixed-type array. The difference between a dynamic-type list and a fixed-type (NumPy-style) array is illustrated in [Figure 2-2](#).

At the implementation level, the array essentially contains a single pointer to one contiguous block of data. The Python list, on the other hand, contains a pointer to a block of pointers, each of which in turn points to a full Python object like the Python integer we saw earlier. Again, the advantage of the list is flexibility: because each list element is a full structure containing both data and type information, the list can be filled with data of any desired type. Fixed-type NumPy-style arrays lack this flexibility, but are much more efficient for storing and manipulating data.

각 데이터 항목이 Python 객체로 구성되며, 각각 유형정보, 참조횟수, 기타정보를 포함

⇒ 모든 변수가 같은 타입을 가지는 경우, 정보가 대부분 중복되므로 '고정 유형 배열'에 저장하는 것이 더 효율적일 수 있음



Python 리스트 > 그냥 리스트

Python List > Just a List

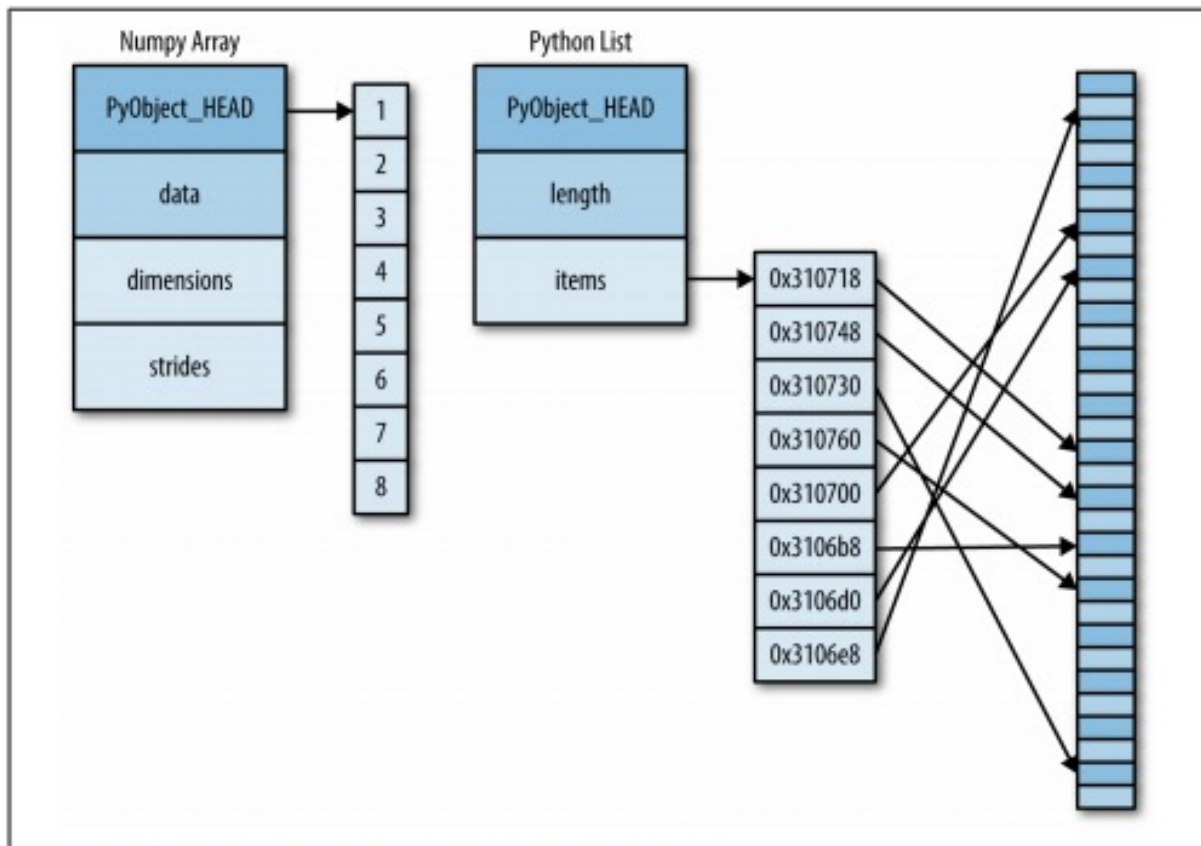


Figure 2-2. The difference between C and Python lists

Python List의 장점

- 유연성 -> 원하는 타입의 데이터로 채워질 수 있다

Numpy Style Array

- 유연성은 낮음
- 데이터 저장 및 조작에 훨씬 더 효율적임



Python의 고정 타입 배열

Fixed-Type Arrays in Python

Python offers several different options for storing data in efficient, fixed-type data buffers. The built-in `array` module (available since Python 3.3) can be used to create dense arrays of a uniform type:

```
In[6]: import array
      L = list(range(10))
      A = array.array('i', L)
      A
```

```
Out[6]: array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Here `'i'` is a type code indicating the contents are integers.

Much more useful, however, is the `ndarray` object of the NumPy package. While Python's `array` object provides efficient storage of array-based data, NumPy adds to this efficient *operations* on that data. We will explore these operations in later sections; here we'll demonstrate several ways of creating a NumPy array.

We'll start with the standard NumPy import, under the alias `np`:

```
In[7]: import numpy as np
```

Python 자체 Array 기능

- 타입 유형코드 + 데이터를 효율적으로 저장

Numpy Array 기능

- 위 기능보다 효율적인 작업들이 추가됨



Python List에서 Array 만들기

Creating Arrays from Python Lists

First, we can use `np.array` to create arrays from Python lists:

```
In[8]: # integer array:  
       np.array([1, 4, 2, 5, 3])
```

```
Out[8]: array([1, 4, 2, 5, 3])
```

Remember that unlike Python lists, NumPy is constrained to arrays that all contain the same type. If types do not match, NumPy will upcast if possible (here, integers are upcast to floating point):

```
In[9]: np.array([3.14, 4, 2, 3])
```

```
Out[9]: array([ 3.14,  4.  ,  2.  ,  3.  ])
```

If we want to explicitly set the data type of the resulting array, we can use the `dtype` keyword:

```
In[10]: np.array([1, 2, 3, 4], dtype='float32')
```

```
Out[10]: array([ 1.,  2.,  3.,  4.], dtype=float32)
```

Finally, unlike Python lists, NumPy arrays can explicitly be multidimensional; here's one way of initializing a multidimensional array using a list of lists:

```
In[11]: # nested lists result in multidimensional arrays  
        np.array([range(i, i + 3) for i in [2, 4, 6]])
```

```
Out[11]: array([[2, 3, 4],  
               [4, 5, 6],  
               [6, 7, 8]])
```

The inner lists are treated as rows of the resulting two-dimensional array.



처음부터 Array 만들기

Creating Arrays from Scratch

Especially for larger arrays, it is more efficient to create arrays from scratch using routines built into NumPy. Here are several examples:

```
In[12]: # Create a length-10 integer array filled with zeros
        np.zeros(10, dtype=int)
```

```
Out[12]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In[13]: # Create a 3x5 floating-point array filled with 1s
        np.ones((3, 5), dtype=float)
```

```
Out[13]: array([[ 1.,  1.,  1.,  1.,  1.],
                 [ 1.,  1.,  1.,  1.,  1.],
                 [ 1.,  1.,  1.,  1.,  1.]])
```

```
Out[14]: array([[ 3.14,  3.14,  3.14,  3.14,  3.14],
                 [ 3.14,  3.14,  3.14,  3.14,  3.14],
                 [ 3.14,  3.14,  3.14,  3.14,  3.14]])
```

```
In[15]: # Create an array filled with a linear sequence
        # Starting at 0, ending at 20, stepping by 2
        # (this is similar to the built-in range() function)
        np.arange(0, 20, 2)
```

```
Out[15]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

```
In[16]: # Create an array of five values evenly spaced between 0 and 1
        np.linspace(0, 1, 5)
```

```
Out[16]: array([ 0. ,  0.25,  0.5 ,  0.75,  1.  ])
```

```
In[17]: # Create a 3x3 array of uniformly distributed
        # random values between 0 and 1
        np.random.random((3, 3))
```

```
Out[17]: array([[ 0.99844933,  0.52183819,  0.22421193],
                 [ 0.08007488,  0.45429293,  0.20941444],
                 [ 0.14360941,  0.96910973,  0.946117  ]])
```

```
In[18]: # Create a 3x3 array of normally distributed random values
        # with mean 0 and standard deviation 1
        np.random.normal(0, 1, (3, 3))
```

```
Out[18]: array([[ 1.51772646,  0.39614948, -0.10634696],
                 [ 0.25671348,  0.00732722,  0.37783601],
                 [ 0.68446945,  0.15926039, -0.70744073]])
```

```
In[19]: # Create a 3x3 array of random integers in the interval [0, 10)
        np.random.randint(0, 10, (3, 3))
```

```
Out[19]: array([[2, 3, 4],
                 [5, 7, 8],
                 [0, 5, 0]])
```

```
In[20]: # Create a 3x3 identity matrix
        np.eye(3)
```

```
Out[20]: array([[ 1.,  0.,  0.],
                 [ 0.,  1.,  0.],
                 [ 0.,  0.,  1.]])
```

```
In[21]: # Create an uninitialized array of three integers
        # The values will be whatever happens to already exist at that
        # memory location
        np.empty(3)
```

```
Out[21]: array([ 1.,  1.,  1.] )
```




Numpy 표준 데이터 타입

NumPy Standard Data Types

NumPy arrays contain values of a single type, so it is important to have detailed knowledge of those types and their limitations. Because NumPy is built in C, the types will be familiar to users of C, Fortran, and other related languages.

The standard NumPy data types are listed in [Table 2-1](#). Note that when constructing an array, you can specify them using a string:

```
np.zeros(10, dtype='int16')
```

Or using the associated NumPy object:

```
np.zeros(10, dtype=np.int16)
```

Table 2-1. Standard NumPy data types

Data type	Description
bool_	Boolean (True or False) stored as a byte
int_	Default integer type (same as C long; normally either int64 or int32)
intc	Identical to C int (normally int32 or int64)
intp	Integer used for indexing (same as C ssize_t; normally either int32 or int64)
int8	Byte (−128 to 127)
int16	Integer (−32768 to 32767)
int32	Integer (−2147483648 to 2147483647)
int64	Integer (−9223372036854775808 to 9223372036854775807)
uint8	Unsigned integer (0 to 255)
uint16	Unsigned integer (0 to 65535)
uint32	Unsigned integer (0 to 4294967295)
uint64	Unsigned integer (0 to 18446744073709551615)
float_	Shorthand for float64
float16	Half-precision float: sign bit, 5 bits exponent, 10 bits mantissa
float32	Single-precision float: sign bit, 8 bits exponent, 23 bits mantissa
float64	Double-precision float: sign bit, 11 bits exponent, 52 bits mantissa
complex_	Shorthand for complex128
complex64	Complex number, represented by two 32-bit floats
complex128	Complex number, represented by two 64-bit floats



Numpy 배열의 기초

The Basics of NumPy Arrays

Data manipulation in Python is nearly synonymous with NumPy array manipulation: even newer tools like Pandas (**Chapter 3**) are built around the NumPy array. This section will present several examples using NumPy array manipulation to access data and subarrays, and to split, reshape, and join the arrays. While the types of operations shown here may seem a bit dry and pedantic, they comprise the building blocks of many other examples used throughout the book. Get to know them well!

We'll cover a few categories of basic array manipulations here:

Attributes of arrays

Determining the size, shape, memory consumption, and data types of arrays

Indexing of arrays

Getting and setting the value of individual array elements

Slicing of arrays

Getting and setting smaller subarrays within a larger array

Reshaping of arrays

Changing the shape of a given array

Joining and splitting of arrays

Combining multiple arrays into one, and splitting one array into many

배열의 속성

- 배열의 크기, 모양, 메모리 소비 및 데이터 유형 결정

배열 인덱싱

- 개별 배열 요소의 값 가져오기 및 설정

배열 슬라이싱

- 더 큰 배열 내에서 더 작은 하위 배열 가져오기 및 설정

배열 재구성

- 주어진 배열의 모양 변경

배열 결합 및 분할

- 여러 배열을 하나로 결합하고 하나의 배열을 여러 배열로 분할



배열 속성 지정

NumPy Array Attributes

First let's discuss some useful array attributes. We'll start by defining three random arrays: a one-dimensional, two-dimensional, and three-dimensional array. We'll use NumPy's random number generator, which we will seed with a set value in order to ensure that the same random arrays are generated each time this code is run:

```
In[1]: import numpy as np
       np.random.seed(0) # seed for reproducibility

       x1 = np.random.randint(10, size=6) # One-dimensional array
       x2 = np.random.randint(10, size=(3, 4)) # Two-dimensional array
       x3 = np.random.randint(10, size=(3, 4, 5)) # Three-dimensional array
```

Each array has attributes `ndim` (the number of dimensions), `shape` (the size of each dimension), and `size` (the total size of the array):

```
In[2]: print("x3.ndim: ", x3.ndim)
       print("x3.shape: ", x3.shape)
       print("x3.size: ", x3.size)

x3.ndim: 3
x3.shape: (3, 4, 5)
x3.size: 60
```

Another useful attribute is the `dtype`, the data type of the array (which we discussed previously in ["Understanding Data Types in Python" on page 34](#)):

```
In[3]: print("dtype:", x3.dtype)
```

```
dtype: int64
```

Other attributes include `itemsize`, which lists the size (in bytes) of each array element, and `nbytes`, which lists the total size (in bytes) of the array:

```
In[4]: print("itemsize:", x3.itemsize, "bytes")
       print("nbytes:", x3.nbytes, "bytes")
```

```
itemsize: 8 bytes
nbytes: 480 bytes
```

In general, we expect that `nbytes` is equal to `itemsize` times `size`.



배열 인덱싱 : 단일 요소 접근하기

Array Indexing: Accessing Single Elements

If you are familiar with Python's standard list indexing, indexing in NumPy will feel quite familiar. In a one-dimensional array, you can access the i^{th} value (counting from zero) by specifying the desired index in square brackets, just as with Python lists:

```
In[5]: x1
Out[5]: array([5, 0, 3, 3, 7, 9])
In[6]: x1[0]
Out[6]: 5
In[7]: x1[4]
Out[7]: 7
```

To index from the end of the array, you can use negative indices:

```
In[8]: x1[-1]
Out[8]: 9
In[9]: x1[-2]
Out[9]: 7
```

In a multidimensional array, you access items using a comma-separated tuple of indices:

```
In[10]: x2
Out[10]: array([[3, 5, 2, 4],
               [7, 6, 8, 8],
               [1, 6, 7, 7]])
In[11]: x2[0, 0]
Out[11]: 3
```

```
In[12]: x2[2, 0]
Out[12]: 1
In[13]: x2[2, -1]
Out[13]: 7
```

You can also modify values using any of the above index notation:

```
In[14]: x2[0, 0] = 12
        x2
Out[14]: array([[12,  5,  2,  4],
               [ 7,  6,  8,  8],
               [ 1,  6,  7,  7]])
```

Keep in mind that, unlike Python lists, NumPy arrays have a fixed type. This means, for example, that if you attempt to insert a floating-point value to an integer array, the value will be silently truncated. Don't be caught unaware by this behavior!

```
In[15]: x1[0] = 3.14159 # this will be truncated!
        x1
Out[15]: array([3, 0, 3, 3, 7, 9])
```



배열 슬라이싱 : 하위 배열 접근하기

Array Slicing: Accessing Subarrays

Just as we can use square brackets to access individual array elements, we can also use them to access subarrays with the *slice* notation, marked by the colon (:) character. The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array `x`, use this:

`x[start:stop:step]`

If any of these are unspecified, they default to the values `start=0`, `stop=size of dimension`, `step=1`. We'll take a look at accessing subarrays in one dimension and in multiple dimensions.

One-dimensional subarrays

```
In[16]: x = np.arange(10)
        x

Out[16]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In[17]: x[:5] # first five elements

Out[17]: array([0, 1, 2, 3, 4])

In[18]: x[5:] # elements after index 5

Out[18]: array([5, 6, 7, 8, 9])

In[19]: x[4:7] # middle subarray

Out[19]: array([4, 5, 6])
```

```
In[20]: x[::2] # every other element

Out[20]: array([0, 2, 4, 6, 8])

In[21]: x[1::2] # every other element, starting at index 1

Out[21]: array([1, 3, 5, 7, 9])
```

A potentially confusing case is when the `step` value is negative. In this case, the defaults for `start` and `stop` are swapped. This becomes a convenient way to reverse an array:

```
In[22]: x[::-1] # all elements, reversed

Out[22]: array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])

In[23]: x[5::-2] # reversed every other from index 5

Out[23]: array([5, 3, 1])
```




배열 슬라이싱 : 하위 배열 접근하기

Array Slicing: Accessing Subarrays

Multidimensional subarrays

Multidimensional slices work in the same way, with multiple slices separated by commas. For example:

```
In[24]: x2
Out[24]: array([[12,  5,  2,  4],
               [ 7,  6,  8,  8],
               [ 1,  6,  7,  7]])

In[25]: x2[:2, :3] # two rows, three columns
Out[25]: array([[12,  5,  2],
               [ 7,  6,  8]])

In[26]: x2[:3, ::2] # all rows, every other column
Out[26]: array([[12,  2],
               [ 7,  8],
               [ 1,  7]])
```

Finally, subarray dimensions can even be reversed together:

```
In[27]: x2[::-1, ::-1]
Out[27]: array([[ 7,  7,  6,  1],
               [ 8,  8,  6,  7],
               [ 4,  2,  5, 12]])
```

Accessing array rows and columns. One commonly needed routine is accessing single rows or columns of an array. You can do this by combining indexing and slicing, using an empty slice marked by a single colon (:):

```
In[28]: print(x2[:, 0]) # first column of x2
[12  7  1]

In[29]: print(x2[0, :]) # first row of x2
[12  5  2  4]
```

In the case of row access, the empty slice can be omitted for a more compact syntax:

```
In[30]: print(x2[0]) # equivalent to x2[0, :]
[12  5  2  4]
```



배열 슬라이싱 : 하위 배열 접근하기

Array Slicing: Accessing Subarrays

Subarrays as no-copy views

One important—and extremely useful—thing to know about array slices is that they return *views* rather than *copies* of the array data. This is one area in which NumPy array slicing differs from Python list slicing: in lists, slices will be copies. Consider our two-dimensional array from before:

```
In[31]: print(x2)

[[12  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

Let's extract a 2x2 subarray from this:

```
In[32]: x2_sub = x2[:2, :2]
        print(x2_sub)

[[12  5]
 [ 7  6]]
```

Now if we modify this subarray, we'll see that the original array is changed! Observe:

```
In[33]: x2_sub[0, 0] = 99
        print(x2_sub)

[[99  5]
 [ 7  6]]

In[34]: print(x2)

[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

This default behavior is actually quite useful: it means that when we work with large datasets, we can access and process pieces of these datasets without the need to copy the underlying data buffer.

Creating copies of arrays

Despite the nice features of array views, it is sometimes useful to instead explicitly copy the data within an array or a subarray. This can be most easily done with the `copy()` method:

```
In[35]: x2_sub_copy = x2[:2, :2].copy()
        print(x2_sub_copy)

[[99  5]
 [ 7  6]]
```

If we now modify this subarray, the original array is not touched:

```
In[36]: x2_sub_copy[0, 0] = 42
        print(x2_sub_copy)

[[42  5]
 [ 7  6]]

In[37]: print(x2)

[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

No copy => 동일 데이터 참조 (수정시 함께 변경)

W/ copy => 다른 데이터 참조 (수정시 단일 변경)



배열 재구조화

Reshaping of Arrays

Another useful type of operation is reshaping of arrays. The most flexible way of doing this is with the `reshape()` method. For example, if you want to put the numbers 1 through 9 in a 3×3 grid, you can do the following:

```
In[38]: grid = np.arange(1, 10).reshape((3, 3))
        print(grid)

[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Note that for this to work, the size of the initial array must match the size of the reshaped array. Where possible, the reshape method will use a no-copy view of the initial array, but with noncontiguous memory buffers this is not always the case.

Another common reshaping pattern is the conversion of a one-dimensional array into a two-dimensional row or column matrix. You can do this with the `reshape` method, or more easily by making use of the `newaxis` keyword within a slice operation:

```
In[39]: x = np.array([1, 2, 3])

        # row vector via reshape
        x.reshape((1, 3))

Out[39]: array([[1, 2, 3]])

In[40]: # row vector via newaxis
        x[np.newaxis, :]

Out[40]: array([[1, 2, 3]])

In[41]: # column vector via reshape
        x.reshape((3, 1))

Out[41]: array([[1],
                [2],
                [3]])

In[42]: # column vector via newaxis
        x[:, np.newaxis]

Out[42]: array([[1],
                [2],
                [3]])
```

We will see this type of transformation often throughout the remainder of the book.



배열 연결 및 분할

Array Concatenation and Splitting

All of the preceding routines worked on single arrays. It's also possible to combine multiple arrays into one, and to conversely split a single array into multiple arrays. We'll take a look at those operations here.

Concatenation of arrays

Concatenation, or joining of two arrays in NumPy, is primarily accomplished through the routines `np.concatenate`, `np.vstack`, and `np.hstack`. `np.concatenate` takes a tuple or list of arrays as its first argument, as we can see here:

```
In[43]: x = np.array([1, 2, 3])
        y = np.array([3, 2, 1])
        np.concatenate([x, y])

Out[43]: array([1, 2, 3, 3, 2, 1])
```

You can also concatenate more than two arrays at once:

```
In[44]: z = [99, 99, 99]
        print(np.concatenate([x, y, z]))

[ 1  2  3  3  2  1 99 99 99]
```

`np.concatenate` can also be used for two-dimensional arrays:

```
In[45]: grid = np.array([[1, 2, 3],
                        [4, 5, 6]])

In[46]: # concatenate along the first axis
        np.concatenate([grid, grid])

Out[46]: array([[1, 2, 3],
                [4, 5, 6],
                [1, 2, 3],
                [4, 5, 6]])

In[47]: # concatenate along the second axis (zero-indexed)
        np.concatenate([grid, grid], axis=1)
```

```
Out[47]: array([[1, 2, 3, 1, 2, 3],
                [4, 5, 6, 4, 5, 6]])
```

For working with arrays of mixed dimensions, it can be clearer to use the `np.vstack` (vertical stack) and `np.hstack` (horizontal stack) functions:

```
In[48]: x = np.array([1, 2, 3])
        grid = np.array([[9, 8, 7],
                        [6, 5, 4]])

        # vertically stack the arrays
        np.vstack([x, grid])

Out[48]: array([[1, 2, 3],
                [9, 8, 7],
                [6, 5, 4]])

In[49]: # horizontally stack the arrays
        y = np.array([[99],
                        [99]])
        np.hstack([grid, y])

Out[49]: array([[ 9,  8,  7, 99],
                [ 6,  5,  4, 99]])
```

Similarly, `np.dstack` will stack arrays along the third axis.



Splitting of arrays

The opposite of concatenation is splitting, which is implemented by the functions `np.split`, `np.hsplit`, and `np.vsplit`. For each of these, we can pass a list of indices giving the split points:

```
In[50]: x = [1, 2, 3, 99, 99, 3, 2, 1]
        x1, x2, x3 = np.split(x, [3, 5])
        print(x1, x2, x3)
```

```
[1 2 3] [99 99] [3 2 1]
```

Notice that N split points lead to $N + 1$ subarrays. The related functions `np.hsplit` and `np.vsplit` are similar:

```
In[51]: grid = np.arange(16).reshape((4, 4))
        grid
```

```
Out[51]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11],
                [12, 13, 14, 15]])
```

```
In[52]: upper, lower = np.vsplit(grid, [2])
        print(upper)
        print(lower)
```

```
[[0 1 2 3]
 [4 5 6 7]]
```

```
In[52]: upper, lower = np.vsplit(grid, [2])
        print(upper)
        print(lower)
```

```
[[0 1 2 3]
 [4 5 6 7]]
```

```
[[ 8  9 10 11]
 [12 13 14 15]]
```

```
In[53]: left, right = np.hsplit(grid, [2])
        print(left)
        print(right)
```

```
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

Similarly, `np.dsplit` will split arrays along the third axis.



배열 연산 : 유니버설 함수란?

Computation on NumPy Arrays: Universal Functions

Up until now, we have been discussing some of the basic nuts and bolts of NumPy; in the next few sections, we will dive into the reasons that NumPy is so important in the Python data science world. Namely, it provides an easy and flexible interface to optimized computation with arrays of data.

Computation on NumPy arrays can be very fast, or it can be very slow. The key to making it fast is to use *vectorized* operations, generally implemented through NumPy's *universal functions* (ufuncs). This section motivates the need for NumPy's ufuncs, which can be used to make repeated calculations on array elements much more efficient. It then introduces many of the most common and useful arithmetic ufuncs available in the NumPy package.

빠르게 만드는 핵심

=> Numpy 범용 함수인 ufuncs를 통해 벡터화된 작업을 사용하는 것!



루프는 느리다

The Slowness of Loops

Python's default implementation (known as CPython) does some operations very slowly. This is in part due to the dynamic, interpreted nature of the language: the fact that types are flexible, so that sequences of operations cannot be compiled down to efficient machine code as in languages like C and Fortran. Recently there have been various attempts to address this weakness: well-known examples are the **PyPy project**, a just-in-time compiled implementation of Python; the **Cython project**, which converts Python code to compilable C code; and the **Numba project**, which converts snippets of Python code to fast LLVM bytecode. Each of these has its strengths and weaknesses, but it is safe to say that none of the three approaches has yet surpassed the reach and popularity of the standard CPython engine.

Python의 기본 구현(CPython이라고 함)은 일부 작업을 매우 느리게 수행

=> 유형이 유연하여 작업 시퀀스를 C 및 Fortran과 같은 언어와 같이 효율적인 기계 코드로 컴파일할 수 없기 때문

해결점 3가지, 하지만 아직 python 기본(Cpython) 활용 범위와 인기를 증가하지는 못함

- Python의 JIT 컴파일 구현인 PyPy 프로젝트
- Python 코드를 컴파일 가능한 C 코드로 변환하는 Cython 프로젝트
- Python 코드 조각을 빠른 LLVM 바이트 코드로 변환하는 Numba 프로젝트



루프는 느리다

The Slowness of Loops

The relative sluggishness of Python generally manifests itself in situations where many small operations are being repeated—for instance, looping over arrays to operate on each element. For example, imagine we have an array of values and we'd like to compute the reciprocal of each. A straightforward approach might look like this:

```
In[1]: import numpy as np
       np.random.seed(0)

       def compute_reciprocals(values):
           output = np.empty(len(values))
           for i in range(len(values)):
               output[i] = 1.0 / values[i]
           return output

       values = np.random.randint(1, 10, size=5)
       compute_reciprocals(values)

Out[1]: array([ 0.16666667,  1.         ,  0.25        ,  0.25        ,  0.125        ])
```

This implementation probably feels fairly natural to someone from, say, a C or Java background. But if we measure the execution time of this code for a large input, we see that this operation is very slow, perhaps surprisingly so! We'll benchmark this with IPython's `%timeit` magic (discussed in [“Profiling and Timing Code” on page 25](#)):

```
In[2]: big_array = np.random.randint(1, 100, size=1000000)
       %timeit compute_reciprocals(big_array)

1 loop, best of 3: 2.91 s per loop
```

It takes several seconds to compute these million operations and to store the result! When even cell phones have processing speeds measured in Giga-FLOPS (i.e., billions of numerical operations per second), this seems almost absurdly slow. It turns out that the bottleneck here is not the operations themselves, but the type-checking and function dispatches that CPython must do at each cycle of the loop. Each time the reciprocal is computed, Python first examines the object's type and does a dynamic lookup of the correct function to use for that type. If we were working in compiled code instead, this type specification would be known before the code executes and the result could be computed much more efficiently.

속도 저하는 대부분 반복문에서 발생

병목 현상은 작업 자체가 아니라 CPython이 루프의 각 주기에서 수행해야 하는 유형 검사 및 함수 디스패치에 있음



Ufuncs 소개

Introducing UFuncs

For many types of operations, NumPy provides a convenient interface into just this kind of statically typed, compiled routine. This is known as a *vectorized* operation. You can accomplish this by simply performing an operation on the array, which will then be applied to each element. This vectorized approach is designed to push the loop into the compiled layer that underlies NumPy, leading to much faster execution.

Compare the results of the following two:

```
In[3]: print(compute_reciprocals(values))
       print(1.0 / values)

[ 0.16666667  1.          0.25         0.25         0.125        ]
[ 0.16666667  1.          0.25         0.25         0.125        ]
```

Looking at the execution time for our big array, we see that it completes orders of magnitude faster than the Python loop:

```
In[4]: %timeit (1.0 / big_array)

100 loops, best of 3: 4.6 ms per loop
```

Vectorized operations in NumPy are implemented via *ufuncs*, whose main purpose is to quickly execute repeated operations on values in NumPy arrays. Ufuncs are extremely flexible—before we saw an operation between a scalar and an array, but we can also operate between two arrays:

```
In[5]: np.arange(5) / np.arange(1, 6)

Out[5]: array([ 0.          ,  0.5          ,  0.66666667,  0.75          ,  0.8          ])
```

And ufunc operations are not limited to one-dimensional arrays—they can act on multidimensional arrays as well:

```
In[6]: x = np.arange(9).reshape((3, 3))
       2 ** x

Out[6]: array([[ 1,  2,  4],
               [ 8, 16, 32],
               [64, 128, 256]])
```

Computations using vectorization through ufuncs are nearly always more efficient than their counterpart implemented through Python loops, especially as the arrays grow in size. Any time you see such a loop in a Python script, you should consider whether it can be replaced with a vectorized expression.

벡터화된 접근 방식은 루프를 NumPy의 기반이 되는 컴파일된 레이어로 푸시하여 훨씬 더 빠른 실행을 유도하도록 설계됨

=> ufunc를 통한 벡터화를 사용한 계산은 특히 배열의 크기가 커질수록 Python 루프를 통해 구현된 계산보다 거의 항상 더 효율적이므로 루프를 볼 때마다 벡터화가 가능한지 여부를 꼭 고려할 것!



NumPy 유니버설 함수(UFuncs)

Exploring NumPy's UFuncs

Ufuncs exist in two flavors: *unary ufuncs*, which operate on a single input, and *binary ufuncs*, which operate on two inputs. We'll see examples of both these types of functions here.

Array arithmetic

NumPy's ufuncs feel very natural to use because they make use of Python's native arithmetic operators. The standard addition, subtraction, multiplication, and division can all be used:

```
In[7]: x = np.arange(4)
print("x      =", x)
print("x + 5 =", x + 5)
print("x - 5 =", x - 5)
print("x * 2 =", x * 2)
print("x / 2 =", x / 2)
print("x // 2 =", x // 2) # floor division
```

```
x      = [0 1 2 3]
x + 5 = [5 6 7 8]
x - 5 = [-5 -4 -3 -2]
x * 2 = [0 2 4 6]
x / 2 = [ 0.   0.5  1.   1.5]
x // 2 = [0 0 1 1]
```

Table 2-2. Arithmetic operators implemented in NumPy

Operator	Equivalent ufunc	Description
+	np.add	Addition (e.g., 1 + 1 = 2)
-	np.subtract	Subtraction (e.g., 3 - 2 = 1)
-	np.negative	Unary negation (e.g., -2)
*	np.multiply	Multiplication (e.g., 2 * 3 = 6)
/	np.divide	Division (e.g., 3 / 2 = 1.5)
//	np.floor_divide	Floor division (e.g., 3 // 2 = 1)
**	np.power	Exponentiation (e.g., 2 ** 3 = 8)
%	np.mod	Modulus/remainder (e.g., 9 % 4 = 1)

There is also a unary ufunc for negation, a `**` operator for exponentiation, and a `%` operator for modulus:

```
In[8]: print("-x      =", -x)
print("x ** 2 =", x ** 2)
print("x % 2  =", x % 2)
```

```
-x      = [ 0 -1 -2 -3]
x ** 2 = [0 1 4 9]
x % 2  = [0 1 0 1]
```

In addition, these can be strung together however you wish, and the standard order of operations is respected:

```
In[9]: -(0.5*x + 1) ** 2
Out[9]: array([-1.   , -2.25, -4.   , -6.25])
```

All of these arithmetic operations are simply convenient wrappers around specific functions built into NumPy; for example, the `+` operator is a wrapper for the `add` function:

```
In[10]: np.add(x, 2)
Out[10]: array([2, 3, 4, 5])
```



NumPy 유니버설 함수(UFuncs)

Exploring NumPy's UFuncs

Absolute value

Just as NumPy understands Python's built-in arithmetic operators, it also understands Python's built-in absolute value function:

```
In[11]: x = np.array([-2, -1, 0, 1, 2])
        abs(x)
```

```
Out[11]: array([2, 1, 0, 1, 2])
```

The corresponding NumPy ufunc is `np.absolute`, which is also available under the alias `np.abs`:

```
In[12]: np.absolute(x)
```

```
Out[12]: array([2, 1, 0, 1, 2])
```

```
In[13]: np.abs(x)
```

```
Out[13]: array([2, 1, 0, 1, 2])
```

This ufunc can also handle complex data, in which the absolute value returns the magnitude:

```
In[14]: x = np.array([3 - 4j, 4 - 3j, 2 + 0j, 0 + 1j])
        np.abs(x)
```

```
Out[14]: array([ 5.,  5.,  2.,  1.])
```

Trigonometric functions

NumPy provides a large number of useful ufuncs, and some of the most useful for the data scientist are the trigonometric functions. We'll start by defining an array of angles:

```
In[15]: theta = np.linspace(0, np.pi, 3)
```

Now we can compute some trigonometric functions on these values:

```
In[16]: print("theta      = ", theta)
        print("sin(theta) = ", np.sin(theta))
        print("cos(theta) = ", np.cos(theta))
        print("tan(theta) = ", np.tan(theta))
```

```
theta      = [ 0.          1.57079633  3.14159265]
sin(theta) = [ 0.00000000e+00  1.00000000e+00  1.22464680e-16]
cos(theta) = [ 1.00000000e+00  6.12323400e-17 -1.00000000e+00]
tan(theta) = [ 0.00000000e+00  1.63312394e+16 -1.22464680e-16]
```

The values are computed to within machine precision, which is why values that should be zero do not always hit exactly zero. Inverse trigonometric functions are also

```
In[17]: x = [-1, 0, 1]
        print("x          = ", x)
        print("arcsin(x) = ", np.arcsin(x))
        print("arccos(x) = ", np.arccos(x))
        print("arctan(x) = ", np.arctan(x))
```

```
x          = [-1, 0, 1]
arcsin(x) = [-1.57079633  0.          1.57079633]
arccos(x) = [ 3.14159265  1.57079633  0.          ]
arctan(x) = [-0.78539816  0.          0.78539816]
```




NumPy 유니버설 함수(UFuncs)

Exploring NumPy's UFuncs

Exponents and logarithms

Another common type of operation available in a NumPy ufunc are the exponentials:

```
In[18]: x = [1, 2, 3]
        print("x      =", x)
        print("e^x    =", np.exp(x))
        print("2^x    =", np.exp2(x))
        print("3^x    =", np.power(3, x))

x      = [1, 2, 3]
e^x    = [ 2.71828183  7.3890561 20.08553692]
2^x    = [ 2.  4.  8.]
3^x    = [ 3  9 27]
```

The inverse of the exponentials, the logarithms, are also available. The basic `np.log` gives the natural logarithm; if you prefer to compute the base-2 logarithm or the base-10 logarithm, these are available as well:

```
In[19]: x = [1, 2, 4, 10]
        print("x      =", x)
        print("ln(x)   =", np.log(x))
        print("log2(x)  =", np.log2(x))
        print("log10(x) =", np.log10(x))

x      = [1, 2, 4, 10]
ln(x)   = [ 0.          0.69314718  1.38629436  2.30258509]
log2(x) = [ 0.          1.          2.          3.32192809]
log10(x) = [ 0.          0.30103     0.60205999  1.          ]
```

There are also some specialized versions that are useful for maintaining precision with very small input:

```
In[20]: x = [0, 0.001, 0.01, 0.1]
        print("exp(x) - 1 =", np.expm1(x))
        print("log(1 + x) =", np.log1p(x))

exp(x) - 1 = [ 0.          0.0010005  0.01005017  0.10517092]
log(1 + x) = [ 0.          0.0009995  0.00995033  0.09531018]
```

When `x` is very small, these functions give more precise values than if the raw `np.log` or `np.exp` were used.



NumPy 유니버설 함수(UFuncs)

Exploring NumPy's UFuncs

Specialized ufuncs

NumPy has many more ufuncs available, including hyperbolic trig functions, bitwise arithmetic, comparison operators, conversions from radians to degrees, rounding and remainders, and much more. A look through the NumPy documentation reveals a lot of interesting functionality.

Another excellent source for more specialized and obscure ufuncs is the submodule `scipy.special`. If you want to compute some obscure mathematical function on your data, chances are it is implemented in `scipy.special`. There are far too many functions to list them all, but the following snippet shows a couple that might come up in a statistics context:

```
In[21]: from scipy import special
```

```
In[22]: # Gamma functions (generalized factorials) and related functions
```

```
x = [1, 5, 10]
```

```
print("gamma(x)      =", special.gamma(x))
```

```
print("ln|gamma(x)| =", special.gammaln(x))
```

```
print("beta(x, 2)   =", special.beta(x, 2))
```

```
gamma(x)      = [ 1.00000000e+00  2.40000000e+01  3.62880000e+05]
```

```
ln|gamma(x)| = [ 0.          3.17805383 12.80182748]
```

```
beta(x, 2)   = [ 0.5          0.03333333 0.00909091]
```

```
In[23]: # Error function (integral of Gaussian)
```

```
# its complement, and its inverse
```

```
x = np.array([0, 0.3, 0.7, 1.0])
```

```
print("erf(x)  =", special.erf(x))
```

```
print("erfc(x) =", special.erfc(x))
```

```
print("erfinv(x) =", special.erfinv(x))
```

```
erf(x)  = [ 0.          0.32862676  0.67780119  0.84270079]
```

```
erfc(x) = [ 1.          0.67137324  0.32219881  0.15729921]
```

```
erfinv(x) = [ 0.          0.27246271  0.73286908          inf]
```

There are many, many more ufuncs available in both NumPy and `scipy.special`. Because the documentation of these packages is available online, a web search along the lines of “gamma function python” will generally find the relevant information.



Specifying output

For large calculations, it is sometimes useful to be able to specify the array where the result of the calculation will be stored. Rather than creating a temporary array, you can use this to write computation results directly to the memory location where you'd like them to be. For all ufuncs, you can do this using the out argument of the function:

```
In[24]: x = np.arange(5)
        y = np.empty(5)
        np.multiply(x, 10, out=y)
        print(y)

[ 0. 10. 20. 30. 40.]
```

This can even be used with array views. For example, we can write the results of a computation to every other element of a specified array:

```
In[25]: y = np.zeros(10)
        np.power(2, x, out=y[::2])
        print(y)

[ 1.  0.  2.  0.  4.  0.  8.  0. 16.  0.]
```

If we had instead written `y[::2] = 2 ** x`, this would have resulted in the creation of a temporary array to hold the results of `2 ** x`, followed by a second operation copying those values into the `y` array. This doesn't make much of a difference for such a small computation, but for very large arrays the memory savings from careful use of the `out` argument can be significant.

Aggregates

For binary ufuncs, there are some interesting aggregates that can be computed directly from the object. For example, if we'd like to *reduce* an array with a particular operation, we can use the `reduce` method of any ufunc. A reduce repeatedly applies a given operation to the elements of an array until only a single result remains.

For example, calling `reduce` on the `add` ufunc returns the sum of all elements in the array:

```
In[26]: x = np.arange(1, 6)
        np.add.reduce(x)

Out[26]: 15
```

Similarly, calling `reduce` on the `multiply` ufunc results in the product of all array elements:

```
In[27]: np.multiply.reduce(x)

Out[27]: 120
```

If we'd like to store all the intermediate results of the computation, we can instead use `accumulate`:

```
In[28]: np.add.accumulate(x)

Out[28]: array([ 1,  3,  6, 10, 15])

In[29]: np.multiply.accumulate(x)

Out[29]: array([ 1,  2,  6, 24, 120])
```



Outer products

Finally, any ufunc can compute the output of all pairs of two different inputs using the outer method. This allows you, in one line, to do things like create a multiplication table:

```
In[30]: x = np.arange(1, 6)  
        np.multiply.outer(x, x)
```

```
Out[30]: array([[ 1,  2,  3,  4,  5],  
               [ 2,  4,  6,  8, 10],  
               [ 3,  6,  9, 12, 15],  
               [ 4,  8, 12, 16, 20],  
               [ 5, 10, 15, 20, 25]])
```



집계: 최소값, 최대값, *etc.*

Aggregations: Min, Max, and Everything in Between

Often when you are faced with a large amount of data, a first step is to compute summary statistics for the data in question. Perhaps the most common summary statistics are the mean and standard deviation, which allow you to summarize the “typical” values in a dataset, but other aggregates are useful as well (the sum, product, median, minimum and maximum, quantiles, etc.).

NumPy has fast built-in aggregation functions for working on arrays; we’ll discuss and demonstrate some of them here.

요약 통계 계산

- 합계, 곱, 중앙값, 최소값, 최대값, 분위수 등



배열 값의 합 구하기

Summing the Values in an Array

As a quick example, consider computing the sum of all values in an array. Python itself can do this using the built-in `sum` function:

```
In[1]: import numpy as np
In[2]: L = np.random.random(100)
        sum(L)
Out[2]: 55.61209116604941
```

The syntax is quite similar to that of NumPy's `sum` function, and the result is the same in the simplest case:

```
In[3]: np.sum(L)
Out[3]: 55.612091166049424
```

However, because it executes the operation in compiled code, NumPy's version of the operation is computed much more quickly:

```
In[4]: big_array = np.random.rand(1000000)
        %timeit sum(big_array)
        %timeit np.sum(big_array)

10 loops, best of 3: 104 ms per loop
1000 loops, best of 3: 442 µs per loop
```

Be careful, though: the `sum` function and the `np.sum` function are not identical, which can sometimes lead to confusion! In particular, their optional arguments have different meanings, and `np.sum` is aware of multiple array dimensions, as we will see in the following section.



최소값 & 최대값

Computation on NumPy Arrays: Universal Functions

Similarly, Python has built-in min and max functions, used to find the minimum value and maximum value of any given array:

```
In[5]: min(big_array), max(big_array)
```

```
Out[5]: (1.1717128136634614e-06, 0.9999976784968716)
```

NumPy's corresponding functions have similar syntax, and again operate much more quickly:

```
In[6]: np.min(big_array), np.max(big_array)
```

```
Out[6]: (1.1717128136634614e-06, 0.9999976784968716)
```

```
In[7]: %timeit min(big_array)
       %timeit np.min(big_array)
```

```
10 loops, best of 3: 82.3 ms per loop
```

```
1000 loops, best of 3: 497 µs per loop
```

For min, max, sum, and several other NumPy aggregates, a shorter syntax is to use methods of the array object itself:

```
In[8]: print(big_array.min(), big_array.max(), big_array.sum())
```

```
1.17171281366e-06 0.999997678497 499911.628197
```

Whenever possible, make sure that you are using the NumPy version of these aggregates when operating on NumPy arrays!



다차원 집계

Multidimensional aggregates

One common type of aggregation operation is an aggregate along a row or column. Say you have some data stored in a two-dimensional array:

```
In[9]: M = np.random.random((3, 4))
       print(M)

[[ 0.8967576  0.03783739  0.75952519  0.06682827]
 [ 0.8354065  0.99196818  0.19544769  0.43447084]
 [ 0.66859307  0.15038721  0.37911423  0.6687194  ]]
```

By default, each NumPy aggregation function will return the aggregate over the entire array:

```
In[10]: M.sum()

Out[10]: 6.0850555667307118
```

Aggregation functions take an additional argument specifying the *axis* along which the aggregate is computed. For example, we can find the minimum value within each column by specifying axis=0:

```
In[11]: M.min(axis=0)

Out[11]: array([ 0.66859307,  0.03783739,  0.19544769,  0.06682827])
```

The function returns four values, corresponding to the four columns of numbers.

Similarly, we can find the maximum value within each row:

```
In[12]: M.max(axis=1)

Out[12]: array([ 0.8967576 ,  0.99196818,  0.6687194  ])
```

The way the axis is specified here can be confusing to users coming from other languages. The `axis` keyword specifies the *dimension of the array that will be collapsed*, rather than the dimension that will be returned. So specifying `axis=0` means that the first axis will be collapsed: for two-dimensional arrays, this means that values within each column will be aggregated.



기타 집계 함수

Other aggregation functions

NumPy provides many other aggregation functions, but we won't discuss them in detail here. Additionally, most aggregates have a NaN-safe counterpart that computes the result while ignoring missing values, which are marked by the special IEEE floating-point NaN value (for a fuller discussion of missing data, see “[Handling Missing Data](#)” on page 119). Some of these NaN-safe functions were not added until NumPy 1.8, so they will not be available in older NumPy versions.

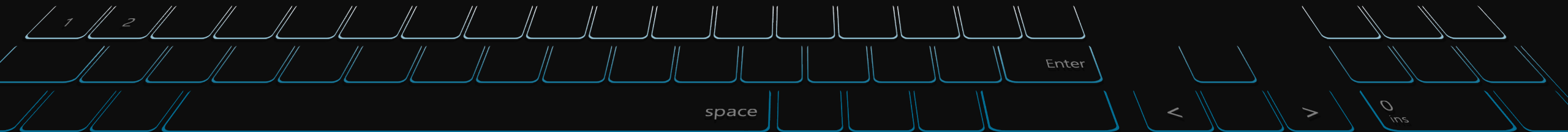
Table 2-3 provides a list of useful aggregation functions available in NumPy.

Table 2-3. Aggregation functions available in NumPy

Function Name	NaN-safe Version	Description
<code>np.sum</code>	<code>np.nansum</code>	Compute sum of elements
<code>np.prod</code>	<code>np.nanprod</code>	Compute product of elements
<code>np.mean</code>	<code>np.nanmean</code>	Compute median of elements
<code>np.std</code>	<code>np.nanstd</code>	Compute standard deviation
<code>np.var</code>	<code>np.nanvar</code>	Compute variance
<code>np.min</code>	<code>np.nanmin</code>	Find minimum value
<code>np.max</code>	<code>np.nanmax</code>	Find maximum value
<code>np.argmin</code>	<code>np.nanargmin</code>	Find index of minimum value
<code>np.argmax</code>	<code>np.nanargmax</code>	Find index of maximum value
<code>np.median</code>	<code>np.nanmedian</code>	Compute median of elements
<code>np.percentile</code>	<code>np.nanpercentile</code>	Compute rank-based statistics of elements
<code>np.any</code>	N/A	Evaluate whether any elements are true
<code>np.all</code>	N/A	Evaluate whether all elements are true

Q&A and Break Time

질의응답 및 휴식시간 (5분)



2부: *Numpy* 중급

- 1) 브로드캐스팅
- 2) 비교, 마스크, 부울 로직
- 3) 팬시 인덱싱
- 4) 구조화된 배열





배열 연산 : 브로드캐스팅 소개

Introducing Broadcasting

We saw in the previous section how NumPy's universal functions can be used to *vectorize* operations and thereby remove slow Python loops. Another means of vectorizing operations is to use NumPy's *broadcasting* functionality. Broadcasting is simply a set of rules for applying binary ufuncs (addition, subtraction, multiplication, etc.) on arrays of different sizes.

Introducing Broadcasting

Recall that for arrays of the same size, binary operations are performed on an element-by-element basis:

```
In[1]: import numpy as np

In[2]: a = np.array([0, 1, 2])
       b = np.array([5, 5, 5])
       a + b

Out[2]: array([5, 6, 7])
```

Broadcasting allows these types of binary operations to be performed on arrays of different sizes—for example, we can just as easily add a scalar (think of it as a zero-dimensional array) to an array:

```
In[3]: a + 5

Out[3]: array([5, 6, 7])
```

We can think of this as an operation that stretches or duplicates the value 5 into the array [5, 5, 5], and adds the results. The advantage of NumPy's broadcasting is that this duplication of values does not actually take place, but it is a useful mental model as we think about broadcasting.

We can similarly extend this to arrays of higher dimension. Observe the result when we add a one-dimensional array to a two-dimensional array:

```
In[4]: M = np.ones((3, 3))
       M

Out[4]: array([[ 1.,  1.,  1.],
               [ 1.,  1.,  1.],
               [ 1.,  1.,  1.]])

In[5]: M + a

Out[5]: array([[ 1.,  2.,  3.],
               [ 1.,  2.,  3.],
               [ 1.,  2.,  3.]])
```

Here the one-dimensional array `a` is stretched, or broadcast, across the second dimension in order to match the shape of `M`.



배열 연산 : 브로드캐스팅 소개

Introducing Broadcasting

While these examples are relatively easy to understand, more complicated cases can involve broadcasting of both arrays. Consider the following example:

```
In[6]: a = np.arange(3)
      b = np.arange(3)[: , np.newaxis]
```

```
print(a)
print(b)
```

```
[0 1 2]
[[0]
 [1]
 [2]]
```

```
In[7]: a + b
```

```
Out[7]: array([[0, 1, 2],
               [1, 2, 3],
               [2, 3, 4]])
```

Just as before we stretched or broadcasted one value to match the shape of the other, here we've stretched *both* a and b to match a common shape, and the result is a two-dimensional array! The geometry of these examples is visualized in Figure 2-4.¹

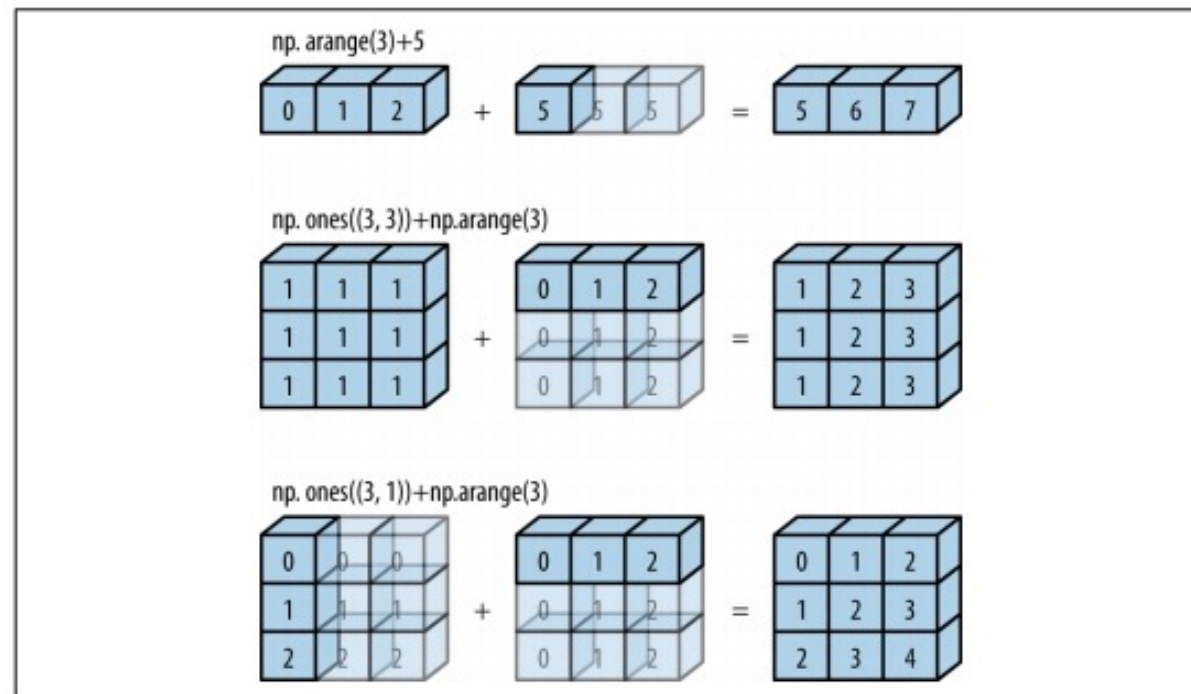


Figure 2-4. Visualization of NumPy broadcasting

The light boxes represent the broadcasted values: again, this extra memory is not actually allocated in the course of the operation, but it can be useful conceptually to imagine that it is.



배열 연산 : 브로드캐스팅 규칙

Rules of Broadcasting

Broadcasting in NumPy follows a strict set of rules to determine the interaction between the two arrays:

- Rule 1: If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is *padded* with ones on its leading (left) side.
- Rule 2: If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape.
- Rule 3: If in any dimension the sizes disagree and neither is equal to 1, an error is raised.

To make these rules clear, let's consider a few examples in detail.

Broadcasting example 1

Let's look at adding a two-dimensional array to a one-dimensional array:

```
In[8]: M = np.ones((2, 3))  
       a = np.arange(3)
```

Let's consider an operation on these two arrays. The shapes of the arrays are:

```
M.shape = (2, 3)  
a.shape = (3,)
```

We see by rule 1 that the array `a` has fewer dimensions, so we pad it on the left with ones:

```
M.shape -> (2, 3)  
a.shape -> (1, 3)
```

By rule 2, we now see that the first dimension disagrees, so we stretch this dimension to match:

```
M.shape -> (2, 3)  
a.shape -> (2, 3)
```

The shapes match, and we see that the final shape will be `(2, 3)`:

```
In[9]: M + a  
Out[9]: array([[ 1.,  2.,  3.],  
               [ 1.,  2.,  3.]])
```



배열 연산 : 브로드캐스팅 규칙

Rules of Broadcasting

Broadcasting example 2

Let's take a look at an example where both arrays need to be broadcast:

```
In[10]: a = np.arange(3).reshape((3, 1))
        b = np.arange(3)
```

Again, we'll start by writing out the shape of the arrays:

```
a.shape = (3, 1)
b.shape = (3,)
```

Rule 1 says we must pad the shape of b with ones:

```
a.shape -> (3, 1)
b.shape -> (1, 3)
```

And rule 2 tells us that we upgrade each of these ones to match the corresponding size of the other array:

```
a.shape -> (3, 3)
b.shape -> (3, 3)
```

Because the result matches, these shapes are compatible. We can see this here:

```
In[11]: a + b
Out[11]: array([[0, 1, 2],
                [1, 2, 3],
                [2, 3, 4]])
```

Broadcasting example 3

Now let's take a look at an example in which the two arrays are not compatible:

```
In[12]: M = np.ones((3, 2))
        a = np.arange(3)
```

This is just a slightly different situation than in the first example: the matrix *M* is transposed. How does this affect the calculation? The shapes of the arrays are:

```
M.shape = (3, 2)
a.shape = (3,)
```

Again, rule 1 tells us that we must pad the shape of a with ones:

```
M.shape -> (3, 2)
a.shape -> (1, 3)
```

By rule 2, the first dimension of a is stretched to match that of *M*:

```
M.shape -> (3, 2)
a.shape -> (3, 3)
```

Now we hit rule 3—the final shapes do not match, so these two arrays are incompatible, as we can observe by attempting this operation:

```
In[13]: M + a
-----

ValueError                                Traceback (most recent call last)

<ipython-input-13-9e16e9f98da6> in <module>()
----> 1 M + a

ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```



실전 브로드캐스팅 : 배열을 중앙 정렬하기

Computation on NumPy Arrays: Universal Functions

Broadcasting operations form the core of many examples we'll see throughout this book. We'll now take a look at a couple simple examples of where they can be useful.

Centering an array

In the previous section, we saw that ufuncs allow a NumPy user to remove the need monly seen example is centering an array of data. Imagine you have an array of 10 observations, each of which consists of 3 values. Using the standard convention (see “Data Representation in Scikit-Learn” on page 343), we'll store this in a 10×3 array:

```
In[17]: X = np.random.random((10, 3))
```

We can compute the mean of each feature using the mean aggregate across the first dimension:

```
In[18]: Xmean = X.mean(0)
        Xmean
```

```
Out[18]: array([ 0.53514715,  0.66567217,  0.44385899])
```

And now we can center the X array by subtracting the mean (this is a broadcasting operation):

```
In[19]: X_centered = X - Xmean
```

To double-check that we've done this correctly, we can check that the centered array has near zero mean:

```
In[20]: X_centered.mean(0)
```

```
Out[20]: array([ 2.22044605e-17, -7.77156117e-17, -1.66533454e-17])
```

To within-machine precision, the mean is now zero.



실전 브로드캐스팅 : 2차원 함수 플로팅하기

Computation on NumPy Arrays: Universal Functions

One place that broadcasting is very useful is in displaying images based on two-dimensional functions. If we want to define a function $z = f(x, y)$, broadcasting can be used to compute the function across the grid:

```
In[21]: # x and y have 50 steps from 0 to 5
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 50)[: , np.newaxis]

z = np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

We'll use Matplotlib to plot this two-dimensional array (these tools will be discussed in full in “Density and Contour Plots” on page 241):

```
In[22]: %matplotlib inline
import matplotlib.pyplot as plt

In[23]: plt.imshow(z, origin='lower', extent=[0, 5, 0, 5],
                  cmap='viridis')
plt.colorbar();
```

The result, shown in Figure 2-5, is a compelling visualization of the two-dimensional function.

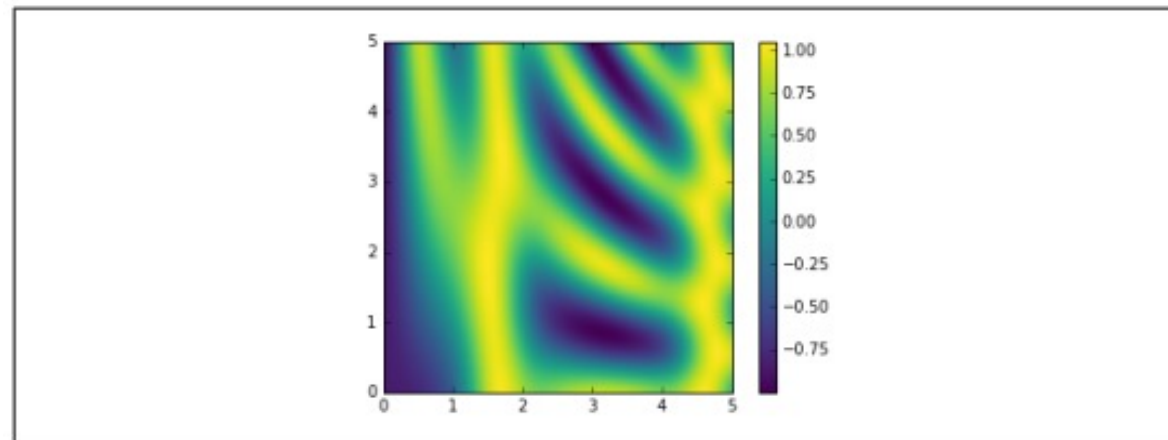


Figure 2-5. Visualization of a 2D array



비교, 마스크, 부울 로직

Comparisons, Masks, and Boolean Logic

This section covers the use of Boolean masks to examine and manipulate values within NumPy arrays. Masking comes up when you want to extract, modify, count, or otherwise manipulate values in an array based on some criterion: for example, you might wish to count all values greater than a certain value, or perhaps remove all outliers that are above some threshold. In NumPy, Boolean masking is often the most efficient way to accomplish these types of tasks.

Example: Counting Rainy Days

Imagine you have a series of data that represents the amount of precipitation each day for a year in a given city. For example, here we'll load the daily rainfall statistics for the city of Seattle in 2014, using Pandas (which is covered in more detail in [Chapter 3](#)):

```
In[1]: import numpy as np
import pandas as pd

# use Pandas to extract rainfall inches as a NumPy array
rainfall = pd.read_csv('data/Seattle2014.csv')['PRCP'].values
inches = rainfall / 254 # 1/10mm -> inches
inches.shape
```

Out[1]: (365,)

The array contains 365 values, giving daily rainfall in inches from January 1 to December 31, 2014.

As a first quick visualization, let's look at the histogram of rainy days shown in [Figure 2-6](#), which was generated using Matplotlib (we will explore this tool more fully in [Chapter 4](#)):

```
In[2]: %matplotlib inline
import matplotlib.pyplot as plt
import seaborn; seaborn.set() # set plot styles

In[3]: plt.hist(inches, 40);
```

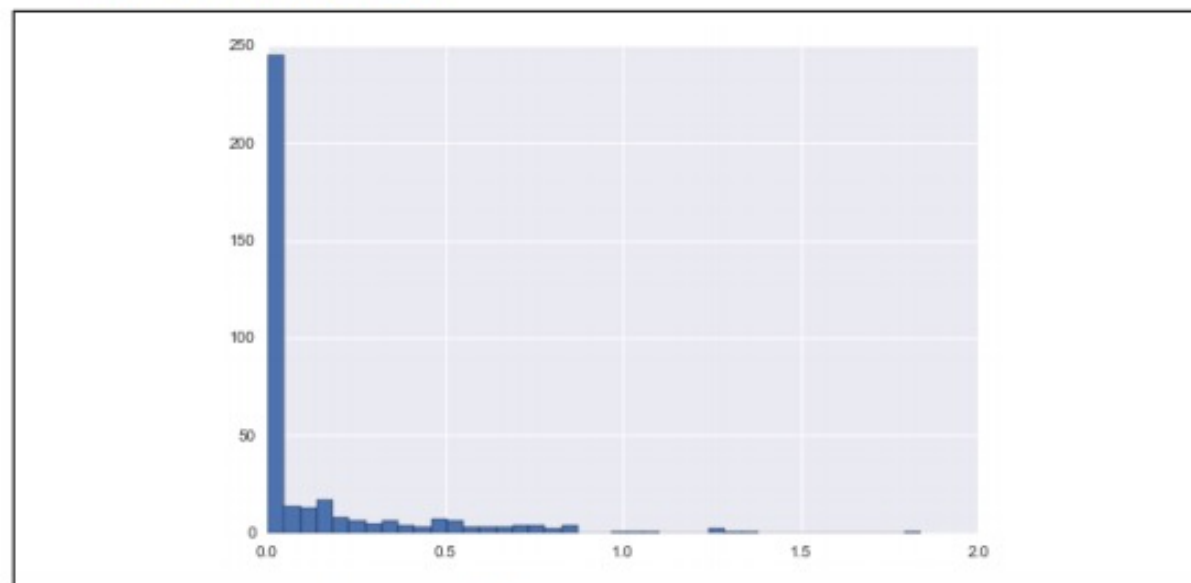


Figure 2-6. Histogram of 2014 rainfall in Seattle

This histogram gives us a general idea of what the data looks like: despite its reputation, the vast majority of days in Seattle saw near zero measured rainfall in 2014. But this doesn't do a good job of conveying some information we'd like to see: for example, how many rainy days were there in the year? What is the average precipitation on those rainy days? How many days were there with more than half an inch of rain?



Ufunc의 비교 연산자

Comparison Operators as ufuncs

In “[Computation on NumPy Arrays: Universal Functions](#)” on page 50 we introduced ufuncs, and focused in particular on arithmetic operators. We saw that using `+`, `-`, `*`, `/`, and others on arrays leads to element-wise operations. NumPy also implements comparison operators such as `<` (less than) and `>` (greater than) as element-wise ufuncs. The result of these comparison operators is always an array with a Boolean data type. All six of the standard comparison operations are available:

```
In[4]: x = np.array([1, 2, 3, 4, 5])
In[5]: x < 3 # less than
Out[5]: array([ True,  True, False, False, False], dtype=bool)
In[6]: x > 3 # greater than
Out[6]: array([False, False, False,  True,  True], dtype=bool)
In[7]: x <= 3 # less than or equal
Out[7]: array([ True,  True,  True, False, False], dtype=bool)
In[8]: x >= 3 # greater than or equal
Out[8]: array([False, False,  True,  True,  True], dtype=bool)
In[9]: x != 3 # not equal
Out[9]: array([ True,  True, False,  True,  True], dtype=bool)
In[10]: x == 3 # equal
Out[10]: array([False, False,  True, False, False], dtype=bool)
```

Operator	Equivalent ufunc
<code>==</code>	<code>np.equal</code>
<code>!=</code>	<code>np.not_equal</code>
<code><</code>	<code>np.less</code>
<code><=</code>	<code>np.less_equal</code>
<code>></code>	<code>np.greater</code>
<code>>=</code>	<code>np.greater_equal</code>

It is also possible to do an element-by-element comparison of two arrays, and to include compound expressions:

```
In[11]: (2 * x) == (x ** 2)
Out[11]: array([False,  True, False, False, False], dtype=bool)
```

As in the case of arithmetic operators, the comparison operators are implemented as ufuncs in NumPy; for example, when you write `x < 3`, internally NumPy uses `np.less(x, 3)`. A summary of the comparison operators and their equivalent ufunc is shown here:

Just as in the case of arithmetic ufuncs, these will work on arrays of any size and shape. Here is a two-dimensional example:

```
In[12]: rng = np.random.RandomState(0)
        x = rng.randint(10, size=(3, 4))
        x
Out[12]: array([[5, 0, 3, 3],
               [7, 9, 3, 5],
               [2, 4, 7, 6]])
In[13]: x < 6
Out[13]: array([[ True,  True,  True,  True],
               [False, False,  True,  True],
               [ True,  True, False, False]], dtype=bool)
```

In each case, the result is a Boolean array, and NumPy provides a number of straightforward patterns for working with these Boolean results.



부울 배열로 작업하기

Working with Boolean Arrays

Given a Boolean array, there are a host of useful operations you can do. We'll work with `x`, the two-dimensional array we created earlier:

```
In[14]: print(x)
```

```
[[5 0 3 3]
 [7 9 3 5]
 [2 4 7 6]]
```

Counting entries

To count the number of True entries in a Boolean array, `np.count_nonzero` is useful:

```
In[15]: # how many values less than 6?
        np.count_nonzero(x < 6)
```

```
Out[15]: 8
```

We see that there are eight array entries that are less than 6. Another way to get at this information is to use `np.sum`; in this case, False is interpreted as 0, and True is interpreted as 1:

```
In[16]: np.sum(x < 6)
```

```
Out[16]: 8
```

The benefit of `sum()` is that like with other NumPy aggregation functions, this summation can be done along rows or columns as well:

```
In[17]: # how many values less than 6 in each row?
        np.sum(x < 6, axis=1)
```

```
Out[17]: array([4, 2, 2])
```

This counts the number of values less than 6 in each row of the matrix.

If we're interested in quickly checking whether any or all the values are true, we can use (you guessed it) `np.any()` or `np.all()`:

```
In[18]: # are there any values greater than 8?
        np.any(x > 8)
```

```
Out[18]: True
```

```
In[19]: # are there any values less than zero?
        np.any(x < 0)
```

```
Out[19]: False
```

```
In[20]: # are all values less than 10?
        np.all(x < 10)
```

```
Out[20]: True
```

```
In[21]: # are all values equal to 6?
        np.all(x == 6)
```

```
Out[21]: False
```

`np.all()` and `np.any()` can be used along particular axes as well. For example:

```
In[22]: # are all values in each row less than 8?
        np.all(x < 8, axis=1)
```

```
Out[22]: array([ True, False,  True], dtype=bool)
```

Here all the elements in the first and third rows are less than 8, while this is not the case for the second row.

Finally, a quick warning: as mentioned in “Aggregations: Min, Max, and Everything in Between” on page 58, Python has built-in `sum()`, `any()`, and `all()` functions. These have a different syntax than the NumPy versions, and in particular will fail or produce unintended results when used on multidimensional arrays. Be sure that you are using `np.sum()`, `np.any()`, and `np.all()` for these examples!



마스크로의 부울 배열

Boolean Arrays as Masks

Boolean operators

We've already seen how we might count, say, all days with rain less than four inches, or all days with rain greater than two inches. But what if we want to know about all days with rain less than four inches *and* greater than one inch? This is accomplished through Python's *bitwise logic operators*, `&`, `|`, `^`, and `~`. Like with the standard arithmetic operators, NumPy overloads these as ufuncs that work element-wise on (usually Boolean) arrays.

For example, we can address this sort of compound question as follows:

```
In[23]: np.sum((inches > 0.5) & (inches < 1))
```

```
Out[23]: 29
```

So we see that there are 29 days with rainfall between 0.5 and 1.0 inches.

Note that the parentheses here are important—because of operator precedence rules, with parentheses removed this expression would be evaluated as follows, which results in an error:

```
inches > (0.5 & inches) < 1
```

Using the equivalence of *A AND B* and *NOT (A OR B)* (which you may remember if you've taken an introductory logic course), we can compute the same result in a different manner:

```
In[24]: np.sum(~( (inches <= 0.5) | (inches >= 1) ))
```

```
Out[24]: 29
```

Combining comparison operators and Boolean operators on arrays can lead to a wide range of efficient logical operations.

The following table summarizes the bitwise Boolean operators and their equivalent ufuncs:

Operator	Equivalent ufunc
<code>&</code>	<code>np.bitwise_and</code>
<code> </code>	<code>np.bitwise_or</code>
<code>^</code>	<code>np.bitwise_xor</code>
<code>~</code>	<code>np.bitwise_not</code>

Using these tools, we might start to answer the types of questions we have about our weather data. Here are some examples of results we can compute when combining masking with aggregations:

```
In[25]: print("Number days without rain: ", np.sum(inches == 0))
        print("Number days with rain: ", np.sum(inches != 0))
        print("Days with more than 0.5 inches:", np.sum(inches > 0.5))
        print("Rainy days with < 0.1 inches :", np.sum((inches > 0) &
                                                         (inches < 0.2)))
```

```
Number days without rain:      215
Number days with rain:        150
Days with more than 0.5 inches: 37
Rainy days with < 0.1 inches : 75
```




마스크로의 부울 배열

Boolean Arrays as Masks

In the preceding section, we looked at aggregates computed directly on Boolean arrays. A more powerful pattern is to use Boolean arrays as masks, to select particular subsets of the data themselves. Returning to our `x` array from before, suppose we want an array of all values in the array that are less than, say, 5:

```
In[26]: x
Out[26]: array([[5, 0, 3, 3],
               [7, 9, 3, 5],
               [2, 4, 7, 6]])
```

We can obtain a Boolean array for this condition easily, as we've already seen:

```
In[27]: x < 5
Out[27]: array([[False,  True,  True,  True],
               [False, False,  True, False],
               [ True,  True, False, False]], dtype=bool)
```

Now to *select* these values from the array, we can simply index on this Boolean array; this is known as a *masking* operation:

```
In[28]: x[x < 5]
Out[28]: array([0, 3, 3, 3, 2, 4])
```

What is returned is a one-dimensional array filled with all the values that meet this condition; in other words, all the values in positions at which the mask array is True.

What is returned is a one-dimensional array filled with all the values that meet this condition; in other words, all the values in positions at which the mask array is True.

We are then free to operate on these values as we wish. For example, we can compute some relevant statistics on our Seattle rain data:

```
In[29]:
# construct a mask of all rainy days
rainy = (inches > 0)

# construct a mask of all summer days (June 21st is the 172nd day)
summer = (np.arange(365) - 172 < 90) & (np.arange(365) - 172 > 0)

print("Median precip on rainy days in 2014 (inches): ",
      np.median(inches[rainy]))
print("Median precip on summer days in 2014 (inches): ",
      np.median(inches[summer]))
print("Maximum precip on summer days in 2014 (inches): ",
      np.max(inches[summer]))
print("Median precip on non-summer rainy days (inches):",
      np.median(inches[rainy & ~summer]))
```

```
Median precip on rainy days in 2014 (inches):    0.194881889764
Median precip on summer days in 2014 (inches):    0.0
Maximum precip on summer days in 2014 (inches):  0.850393700787
Median precip on non-summer rainy days (inches): 0.200787401575
```

By combining Boolean operations, masking operations, and aggregates, we can very quickly answer these sorts of questions for our dataset.



팬시 인덱싱 알아보기

Exploring Fancy Indexing

In the previous sections, we saw how to access and modify portions of arrays using simple indices (e.g., `arr[0]`), slices (e.g., `arr[:5]`), and Boolean masks (e.g., `arr[arr > 0]`). In this section, we'll look at another style of array indexing, known as *fancy indexing*. Fancy indexing is like the simple indexing we've already seen, but we pass arrays of indices in place of single scalars. This allows us to very quickly access and modify complicated subsets of an array's values.

Exploring Fancy Indexing

Fancy indexing is conceptually simple: it means passing an array of indices to access multiple array elements at once. For example, consider the following array:

```
In[1]: import numpy as np
      rand = np.random.RandomState(42)

      x = rand.randint(100, size=10)
      print(x)

[51 92 14 71 60 20 82 86 74 74]
```

Suppose we want to access three different elements. We could do it like this:

```
In[2]: x[3], x[7], x[2]]

Out[2]: [71, 86, 14]
```

Alternatively, we can pass a single list or array of indices to obtain the same result:

```
In[3]: ind = [3, 7, 4]
      x[ind]

Out[3]: array([71, 86, 60])
```

With fancy indexing, the shape of the result reflects the shape of the *index arrays* rather than the shape of the *array being indexed*:

```
In[4]: ind = np.array([[3, 7],
                      [4, 5]])
      x[ind]

Out[4]: array([[71, 86],
              [60, 20]])
```

Fancy indexing also works in multiple dimensions. Consider the following array:

```
In[5]: X = np.arange(12).reshape((3, 4))
      X

Out[5]: array([[ 0,  1,  2,  3],
              [ 4,  5,  6,  7],
              [ 8,  9, 10, 11]])
```

Like with standard indexing, the first index refers to the row, and the second to the column:

```
In[6]: row = np.array([0, 1, 2])
      col = np.array([2, 1, 3])
      X[row, col]

Out[6]: array([ 2,  5, 11])
```

Notice that the first value in the result is `X[0, 2]`, the second is `X[1, 1]`, and the third is `X[2, 3]`. The pairing of indices in fancy indexing follows all the broadcasting rules that were mentioned in [“Computation on Arrays: Broadcasting” on page 63](#). So,



팬시 인덱싱 알아보기

Exploring Fancy Indexing

With fancy indexing, the shape of the result reflects the shape of the *index arrays* rather than the shape of the *array being indexed*:

```
In[4]: ind = np.array([[3, 7],  
                       [4, 5]])  
       x[ind]  
Out[4]: array([[71, 86],  
              [60, 20]])
```

Fancy indexing also works in multiple dimensions. Consider the following array:

```
In[5]: X = np.arange(12).reshape((3, 4))  
       X  
Out[5]: array([[ 0,  1,  2,  3],  
              [ 4,  5,  6,  7],  
              [ 8,  9, 10, 11]])
```

Like with standard indexing, the first index refers to the row, and the second to the column:

```
In[6]: row = np.array([0, 1, 2])  
       col = np.array([2, 1, 3])  
       X[row, col]  
Out[6]: array([ 2,  5, 11])
```

Notice that the first value in the result is $X[0, 2]$, the second is $X[1, 1]$, and the third is $X[2, 3]$. The pairing of indices in fancy indexing follows all the broadcasting rules that were mentioned in “[Computation on Arrays: Broadcasting](#)” on page 63. So,

for example, if we combine a column vector and a row vector within the indices, we get a two-dimensional result:

```
In[7]: X[row[:, np.newaxis], col]  
Out[7]: array([[ 2,  1,  3],  
              [ 6,  5,  7],  
              [10,  9, 11]])
```

Here, each row value is matched with each column vector, exactly as we saw in broadcasting of arithmetic operations. For example:

```
In[8]: row[:, np.newaxis] * col  
Out[8]: array([[0, 0, 0],  
              [2, 1, 3],  
              [4, 2, 6]])
```

It is always important to remember with fancy indexing that the return value reflects the *broadcasted shape of the indices*, rather than the shape of the array being indexed.



결합 인덱싱

Combined Indexing

For even more powerful operations, fancy indexing can be combined with the other indexing schemes we've seen:

```
In[9]: print(X)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

We can combine fancy and simple indices:

```
In[10]: X[2, [2, 0, 1]]
```

```
Out[10]: array([10,  8,  9])
```

We can also combine fancy indexing with slicing:

```
In[11]: X[1:, [2, 0, 1]]
```

```
Out[11]: array([[ 6,  4,  5],
                [10,  8,  9]])
```

And we can combine fancy indexing with masking:

```
In[12]: mask = np.array([1, 0, 1, 0], dtype=bool)
        X[row[:, np.newaxis], mask]
```

```
Out[12]: array([[ 0,  2],
                [ 4,  6],
                [ 8, 10]])
```

All of these indexing options combined lead to a very flexible set of operations for accessing and modifying array values.



팬시 인덱싱으로 값 변경하기

Modifying Values with Fancy Indexing

Just as fancy indexing can be used to access parts of an array, it can also be used to modify parts of an array. For example, imagine we have an array of indices and we'd like to set the corresponding items in an array to some value:

```
In[18]: x = np.arange(10)
        i = np.array([2, 1, 8, 4])
        x[i] = 99
        print(x)

[ 0 99 99  3 99  5  6  7 99  9]
```

We can use any assignment-type operator for this. For example:

```
In[19]: x[i] -= 10
        print(x)

[ 0 89 89  3 89  5  6  7 89  9]
```

Notice, though, that repeated indices with these operations can cause some potentially unexpected results. Consider the following:

```
In[20]: x = np.zeros(10)
        x[[0, 0]] = [4, 6]
        print(x)

[ 6.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

Where did the 4 go? The result of this operation is to first assign $x[0] = 4$, followed by $x[0] = 6$. The result, of course, is that $x[0]$ contains the value 6.

Fair enough, but consider this operation:

```
In[21]: i = [2, 3, 3, 4, 4, 4]
        x[i] += 1
        x

Out[21]: array([ 6.,  0.,  1.,  1.,  1.,  0.,  0.,  0.,  0.,  0.])
```

You might expect that $x[3]$ would contain the value 2, and $x[4]$ would contain the value 3, as this is how many times each index is repeated. Why is this not the case? Conceptually, this is because $x[i] += 1$ is meant as a shorthand of $x[i] = x[i] + 1$. $x[i] + 1$ is evaluated, and then the result is assigned to the indices in x . With this in mind, it is not the augmentation that happens multiple times, but the assignment, which leads to the rather nonintuitive results.

So what if you want the other behavior where the operation is repeated? For this, you can use the `at()` method of ufuncs (available since NumPy 1.8), and do the following:

```
In[22]: x = np.zeros(10)
        np.add.at(x, i, 1)
        print(x)

[ 0.  0.  1.  2.  3.  0.  0.  0.  0.  0.]
```

The `at()` method does an in-place application of the given operator at the specified indices (here, i) with the specified value (here, 1). Another method that is similar in spirit is the `reduceat()` method of ufuncs, which you can read about in the NumPy documentation.



Up to this point we have been concerned mainly with tools to access and operate on array data with NumPy. This section covers algorithms related to sorting values in NumPy arrays. These algorithms are a favorite topic in introductory computer science courses: if you've ever taken one, you probably have had dreams (or, depending on your temperament, nightmares) about insertion sorts, selection sorts, merge sorts, quick sorts, bubble sorts, and many, many more. All are means of accomplishing a similar task: sorting the values in a list or array.

For example, a simple *selection sort* repeatedly finds the minimum value from a list, and makes swaps until the list is sorted. We can code this in just a few lines of Python:

```
In[1]: import numpy as np

def selection_sort(x):
    for i in range(len(x)):
        swap = i + np.argmin(x[i:])
        (x[i], x[swap]) = (x[swap], x[i])
    return x

In[2]: x = np.array([2, 1, 4, 3, 5])
       selection_sort(x)

Out[2]: array([1, 2, 3, 4, 5])
```

As any first-year computer science major will tell you, the selection sort is useful for its simplicity, but is much too slow to be useful for larger arrays. For a list of N values, it requires N loops, each of which does on the order of $\sim N$ comparisons to find the swap value. In terms of the “big-O” notation often used to characterize these algorithms (see “[Big-O Notation](#)” on page 92), selection sort averages $\mathcal{O}[N^2]$: if you double the number of items in the list, the execution time will go up by about a factor of four.

Even selection sort, though, is much better than my all-time favorite sorting algorithms, the *bogosort*:

```
In[3]: def bogosort(x):
       while np.any(x[:-1] > x[1:]):
           np.random.shuffle(x)
       return x

In[4]: x = np.array([2, 1, 4, 3, 5])
       bogosort(x)

Out[4]: array([1, 2, 3, 4, 5])
```

This silly sorting method relies on pure chance: it repeatedly applies a random shuffling of the array until the result happens to be sorted. With an average scaling of $\mathcal{O}[N \times N!]$ (that's N times N factorial), this should—quite obviously—never be used for any real computation.



Numpy의 빠른 정렬 : *np.sort* & *np.argsort*

Fast Sorting in NumPy: *np.sort* and *np.argsort*

Although Python has built-in *sort* and *sorted* functions to work with lists, we won't discuss them here because NumPy's *np.sort* function turns out to be much more efficient and useful for our purposes. By default *np.sort* uses an $\mathcal{O}[N \log N]$, *quicksort* algorithm, though *mergesort* and *heapsort* are also available. For most applications, the default quicksort is more than sufficient.

To return a sorted version of the array without modifying the input, you can use *np.sort*:

```
In[5]: x = np.array([2, 1, 4, 3, 5])
       np.sort(x)
```

```
Out[5]: array([1, 2, 3, 4, 5])
```

If you prefer to sort the array in-place, you can instead use the *sort* method of arrays:

```
In[6]: x.sort()
       print(x)
```

```
[1 2 3 4 5]
```

A related function is *argsort*, which instead returns the *indices* of the sorted elements:

```
In[7]: x = np.array([2, 1, 4, 3, 5])
       i = np.argsort(x)
       print(i)
```

```
[1 0 3 2 4]
```

The first element of this result gives the index of the smallest element, the second value gives the index of the second smallest, and so on. These indices can then be used (via fancy indexing) to construct the sorted array if desired:

```
In[8]: x[i]
```

```
Out[8]: array([1, 2, 3, 4, 5])
```



Numpy의 빠른 정렬 : *np.sort* & *np.argsort*

Fast Sorting in NumPy: *np.sort* and *np.argsort*

Sorting along rows or columns

A useful feature of NumPy's sorting algorithms is the ability to sort along specific rows or columns of a multidimensional array using the *axis* argument. For example:

```
In[9]: rand = np.random.RandomState(42)
      X = rand.randint(0, 10, (4, 6))
      print(X)
```

```
[[6 3 7 4 6 9]
 [2 6 7 4 3 7]
 [7 2 5 4 1 7]
 [5 1 4 0 9 5]]
```

```
In[10]: # sort each column of X
        np.sort(X, axis=0)
```

```
Out[10]: array([[2, 1, 4, 0, 1, 5],
               [6, 3, 7, 4, 6, 7],
               [7, 6, 7, 4, 9, 9]])
```

```
In[11]: # sort each row of X
        np.sort(X, axis=1)
```

```
Out[11]: array([[3, 4, 6, 6, 7, 9],
               [2, 3, 4, 6, 7, 7],
               [1, 2, 4, 5, 7, 7],
               [0, 1, 4, 5, 5, 9]])
```

Keep in mind that this treats each row or column as an independent array, and any relationships between the row or column values will be lost!

Q&A

질의응답 (5분)



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

